Basic Datatypes in Java

Type	📒 Lecture
Date	@January 3, 2022
Lecture #	2
Lecture URL	https://youtu.be/IwTdVsLxz04
Notion URL	https://21f1003586.notion.site/Basic-Datatypes-in-Java-3cfdcb7953c14d4890ccc104b8264b26
Week #	2

Scalar Types

- In an object-oriented langauge, all data should be encapsulated as objects
- However, this is cumbersome
  - Useful to manipulate numeric values like conventional languages
- Java has 8 primitive datatypes
  - `int, long, short, byte`
  - `float, double`
  - `char`
  - `boolean`
- Size of each type is fixed by JVM
  - Does not depend on native architecture

Type	Size in bytes
int	4
long	8
short	2
byte	1
float	4
double	8
char	2
boolean	1

- 2-byte `char` for Unicode

Declarations, assigning values

- We declare variables before we use them

```
int x, y;
float y;
char c;
boolean b1;
```

- The assignment statement works as usual

```
int x, y;
x = 5;
y = 7;
```

- Characters are written with single-quotes (only)
  - Double quotes mark string

```
char c, d;
c = ‘x’;
d = ‘\u03C0’;  // Greek pi, unicode
```

- Boolean constants are `true, false`

```
boolean b1, b2;
b1 = false;
b2 = true;
```

Initialization, constants

- Declarations can come anywhere

```
int x;
x = 10;
float y;
```

  - Use this judiciously to retain readability
- Initialize at the time of declaration

```
int x = 10;
float y = 5.7;
```

- Modifier `final` marks as constant

```
final float pi = 3.1415927f;
pi = 22/7;  // Flagged as error
```

Operators, shortcuts, type casting

- Arithmetic operators are the usual ones
  - `+, -, *, /, %`
- No separate integer division operator `//`

- When both arguments are integer, `/` is integer division

  ```
  float f = 22/7;  // Value is 3.0
  ```

  - Note implicit conversion from `int` to `float`
- No exponentiation operator, use `Math.pow()`
- `Math.pow(a, n)` returns a^n
- Special operators for incrementin and decrementing integers

  ```
  int a = 0, b = 10;
  a++;  // Same as a = a + 1
  b–;  // Same as b = b - 1
  ```

- Shortcut for updating a variable

  ```
  int a = 0, b = 10;
  a += 7;  // Same as a = a + 7
  b *= 12;  // Same as b = b * 12
  ```

Strings

- `String` is a built in class

  ```
  String s, t;
  ```

- String constants enclosed in double quotes

  ```
  String s = "Hello", t = "world";
  ```

- `+` is overloaded for string concatenation

  ```
  String s = "Hello";
  String t = "world";
  String u = s + " " + t;  // "Hello world"
  ```

- String are not arrays of characters
  - We cannot write the following

    ```
    s[3] = 'p';
    s[4] = '!';
    ```

- Instead, invoke method substring in class `String`

  ```
  s = s.substring(0, 3) + "p!";
  ```

- If we change a `String`, we get a new object
  - Strings are immutable
  - After the update, `s` points to a new `String`
  - Java does automatic garbage collection

Arrays

- Arrays are also objects
- Typical declaration

  ```
  int[] a;
  a = new int[100];
  ```

  or

  ```
  int[] a = new int[100];
  ```

- `a.length` gives the size of `a`
  - Note, for `String`, it is a method `s.length()`
- Array indices run from `0` to `a.length - 1`
- Size of the array can vary
- Array constants: `{v1, v2, v3}`
- For example

  ```
  int[] a;
  int n;
  n = 10;
  ```

`a = new int[n];`

`n = 20;`

`a = new int[n];`

`a = {2, 3, 5, 7, 11};`

Summary

- Java allows scalar types, which are not objects
  - `int, long, short, byte, float, double, char, boolean`
- Declarations can include initializations
- Strings and arrays are objects